支持向量机 SVM

Module 1 · Topic 4 — 知识点完整整理

ARTI2001 Machine Learning

涵盖：讲座 16 页 · 课程网页四篇阅读材料 · Tutorial 3 题 + 扩展 · Practical 5 题

含完整数学推导（最大间隔 → 规范形式 → 拉格朗日 → 对偶）

配套文件：《M1T4 题库与解析》

目录

1. 路线图与整体框架
2. 感知机与 SVM：共享打分函数，不同目标
3. 间隔几何：四个必须分清的量（含手算例题与间隔约定）
4. **最大间隔的完整推导与对偶形式**（argmax → 规范形式 → 拉格朗日 → 对偶）
5. 可靠的 SVM 工作流程
6. 软间隔与超参数
7. 核函数的选择（含核技巧手算验证）
8. 概念检查点与第一部分总结
9. **其他经典机器学习算法**（朴素贝叶斯 / KNN / 决策树）
10. Tutorial 完整解答（Q1–Q3 + 扩展）
11. Practical 完整解答（Q1–Q5）
12. 附录 A：公式速查表
13. 附录 B：高频易错点清单
14. 附录 C：术语中英对照

第一部分 · 讲座知识点

1. 路线图与整体框架（第 3 页）

本周核心问题：SVM 是怎么挑选决策边界的？答案是通过控制**间隔（margin）**与**违反（violations）**。特征缩放、C、核函数选择与验证方法都会影响最终边界。

阶段	内容
1. Perceptron → SVM	从「任意一个分隔面」进步到「最大间隔分隔面」
2. Margin Geometry	有符号距离（signed distance）与规范化缩放（canonical scaling）
3. Soft Margin	C 平衡间隔宽度与违反程度
4. Dual View	支持向量决定解
5. Kernel Trick	通过相似度（similarity）实现非线性边界
6. Tutorial & Practical	先手算推理，再用 Python 实验

> **一句话串联：**感知机随便找条线 → SVM 要找最宽的那条（需先定义清楚「宽」怎么量）→ 数据分不开就允许违规（用 C 调节）→ 从对偶视角看只有支持向量说了算 → 换个核函数就能画曲线。

课件强调：**顺序不能乱跳**，因为后面的计算假设前面的定义和约定已经清楚。

2. 感知机与 SVM（第 4 页）

2.1 共同点

两者使用**同一个线性打分函数**：

$$f(x) = w^T x + b$$

w 决定边界方向，b 决定边界位置。预测规则相同：看 f(x) 的符号，正判 +1，负判 −1。

> ★ **考点** 变的不是模型形式，变的是**训练目标（the training objective changes）**。同一条直线，两个算法用不同标准去挑它。

2.2 感知机的四个特征

- **Finds a separating boundary** — 找到一条能分开的边界就行
- **Updates after mistakes or ties** — 只在犯错或打平（f(x)=0）时更新
- **Any valid separator may be returned** — 解不唯一，跑两次可能得到不同的线
- **No explicit margin objective** — 没有显式的间隔目标，贴着某点擦过去也满意

2.3 SVM 的四个特征

- **Selects a maximum-margin separator** — 在无数条可行线中挑出间隔最大的，解唯一
- **Hard margin maximises separation** — 硬间隔，前提是**数据完美可分**
- **Soft margin balances width and violations** — 软间隔，通过 C 取平衡（实际使用的版本）
- **Uses support vectors** — 支持向量定义为**具有非零对偶影响（nonzero dual influence）的训练观测**

> **易忽略的提醒：**两种目标**都不保证在测试集上表现稳健**。最大间隔只是训练集上的一个几何标准，不是泛化保证，最终仍要用**验证证据（validation evidence）**说话。

	感知机	SVM
打分函数	$f(x)=w^Tx+b$	同左
目标	分开就行	间隔最大
解唯一吗	不唯一	唯一（凸优化）
关心间隔吗	不关心	这就是它的目标
谁决定边界	所有犯过错的点	只有支持向量

3. 间隔几何（第 5 页）

副标题点题：**距离只有在除以 $\|w\|$ 之后才是几何量（Distance is geometric only after normalising by $\|w\|$）**。

3.1 四个必须分清的量

名称	公式	含义
原始分数 raw score	$f(x)$	未归一化，随 (w,b) 缩放而变
有符号距离 signed distance	$f(x)/\|w\|$	带正负号，保留「在哪一侧」
点到边界距离 point distance	$\|f(x)\|/\|w\|$	非负，纯几何距离
带标签几何间隔	$\gamma_i = y_i(w^Tx_i+b)/\|w\|$	分对为正，分错为负

3.2 如何读 y f(x)

y f(x)	含义（Read the formula）
> 0	分类正确（correct side）
$= 0$	落在决策边界上（on the boundary），**没有严格的类别归属**
$0 < y\,f(x) < 1$	分对了但落在间隔内部（inside the margin）— 属于违规
$= 1$	落在支持超平面上（support hyperplane）

> ★ **考点** $y\,f(x)$ **大于 1 才算完全安全**。大于 0 只说明分对了，不代表没有违规。

3.3 规范化缩放（canonical scaling）

把 (w, b) 同乘任意正数 k，边界不变（$kw^Tx + kb = 0$ 与 $w^Tx + b = 0$ 是同一条线），但 f(x) 变成 k 倍。这个自由度必须固定，否则「间隔多大」无法比较。

规范约定：让离边界最近的点满足 $|f(x)| = 1$，即所有点满足

$$y_i\, f(x_i) \geqslant 1$$

在此约定下：

- 支持超平面为 $y\,f(x) = 1$
- **一侧间隔（one-sided margin）$= 1/\|w\|$**
- **完整间隔宽度（full width）$= 2/\|w\|$**

> ★ **考点** 「最大化间隔」等价于「**最小化 $\|w\|$**」，这就是优化目标里出现 $\frac{1}{2}\|w\|^2$ 的原因。

3.4 Study check：为什么符号会不一样

问：对一个被误分类的点，$f(x)/\|w\|$ 与 $y \cdot f(x)/\|w\|$ 为什么可能符号相反？

答：有符号距离只看点在边界哪一侧，与真实标签无关；带标签间隔把真实标签乘了进去，反映的是「对不对」。

例：$w = [3, 4]^T$，$b = -5$，$\|w\| = 5$，$x = (2, 1)$

- $f(x) = 3 \cdot 2 + 4 \cdot 1 - 5 = \mathbf{5}$

- 有符号距离 = 5/5 = **+1**
- 若真实标签 y = −1：y · f(x) = −5，带标签几何间隔 = **−1**

符号相反，正是误分类的标志。

> **手算固定套路：** ① 算 $f(x) = w^Tx + b$ → ② 算 $\| w \| = \sqrt{(\Sigma w_j^2)}$ → ③ 有符号距离 = $f(x)/\| w \|$ → ④ 间隔分数 = $y \cdot f(x)$ → ⑤ 对照四档表判断。每一步中间量都写出来并标名。

3.5 课程网页手算例题（SVM - margin）

题目： $w = (2, 3)$，$b = -7$，两个点 $x_1 = (3, 2)$ 标签 $y_1 = 1$；$x_2 = (1, 1)$ 标签 $y_2 = -1$。

Step 1 · 计算权重向量的范数

$$\| w \| = \sqrt{(w_1^2 + w_2^2)} = \sqrt{(2^2 + 3^2)} = \sqrt{13} \approx \mathbf{3.605}$$

Step 2 · 计算每个点到决策边界的距离并分类

点到边界的距离公式：

$$d(x_i) = |w^Tx_i + b| \,/\, \| w \|$$

对 $x_1 = (3, 2)$，$y_1 = 1$：

- $w^Tx_1 + b = (2 \cdot 3) + (3 \cdot 2) - 7 = 6 + 6 - 7 = \mathbf{5}$
- $d(x_1) = |5| / \sqrt{13} = 5 / 3.605 \approx \mathbf{1.387}$
- 因为 $5 > 0$，x_1 被正确分类为**正类**（$y_1 = 1$）

对 $x_2 = (1, 1)$，$y_2 = -1$：

- $w^Tx_2 + b = (2 \cdot 1) + (3 \cdot 1) - 7 = 2 + 3 - 7 = \mathbf{-2}$
- $d(x_2) = |-2| / \sqrt{13} = 2 / 3.605 \approx \mathbf{0.555}$
- 因为 $-2 < 0$，x_2 被正确分类为**负类**（$y_2 = -1$）

Step 3 · 解释最小观测距离

两个观测中，$x_2 = (1, 1)$ 的几何距离更小，约 **0.555**。这是**这些样本的最小观测距离（minimum observed distance for these examples）**。

> **★ 重要的间隔约定（Important margin convention）**
>
> 为单个点计算出来的是它到决策边界的**几何距离**。**几个示例点中的最小距离并不自动就是 SVM 的间隔**，除非这些点恰好是一个已拟合 SVM 的支持向量。
>
> 只有当最近的观测在规范化缩放下是支持向量时，才能把这个距离称为**一侧 SVM 间隔**。
>
> 在规范化 SVM 缩放下，边界到任一支持超平面的距离是 **$1/\| w \|$**，两条支持超平面之间的**完整宽度是 $2/\| w \|$**。**永远要说明你报告的是哪一种约定。**

4. 最大间隔的完整推导与对偶形式

这一章对应课程网页 *SVM - margin maximisation via its dual form*，是整个 T4 唯一需要写推导的部分。

4.1 出发点：最大化最小的带标签几何间隔

先把两个量分清（课程网页原话）：

- **有符号距离（Signed distance）**：$f(x_i)/\|w\|$，它的**符号表示点在超平面的哪一侧**。
- **带标签几何间隔（Labelled geometric margin）**：$\gamma_i = y_i f(x_i)/\|w\|$，它的**符号表示该观测对它的标签而言是否在正确的一侧**。

SVM 训练的目标是：在训练观测上最大化最小的带标签几何间隔。写成 argmax 问题：

$$\arg\max_{w,b} \{ (1/\|w\|) \cdot \min_i [\, y_i(w \cdot x_i + b) \,] \}$$

直观含义：造一个决策边界，让它**离两侧最近的数据点尽可能远**，最大化两类之间的分离。

4.2 第一步化简：引入规范形式（canonical form）

上式直接求解很困难。注意到**可以任意重新缩放 w 和 b 而不改变超平面的方向**（因为 $kw \cdot x + kb = 0$ 与 $w \cdot x + b = 0$ 是同一个超平面），我们利用这个自由度，把 w 和 b 缩放成使**最近的数据点（支持向量）的函数间隔恰好为 1**：

$$y_i(w \cdot x_i + b) = 1 \quad \text{（对支持向量）}$$

> ★ **考点** 它们到决策边界的**几何距离因此是 $1/\|w\|$，而不是 1**。这是最容易混淆的一点：函数间隔（functional margin）= 1，几何间隔（geometric margin）= $1/\|w\|$。

于是所有其他数据点必须满足约束：

$$y_i(w \cdot x_i + b) \geqslant 1, \quad i = 1 \cdots N$$

4.3 第二步：目标从「最大化间隔」变成「最小化 $\|w\|$」

在规范形式下，间隔 $= 1/\|w\|$，与 $\|w\|$ 成**反比**。所以最大化间隔 $\Longleftrightarrow$ 最小化 $\|w\|$。正式写成：

$$\min_{w,b} \tfrac{1}{2}\|w\|^2 \quad \text{subject to} \quad y_i(w \cdot x_i + b) \geqslant 1 \quad \forall i$$

为什么是 $\frac{1}{2}\|w\|^2$ 而不是 $\|w\|$？课程网页说明：**$\frac{1}{2}$ 这个因子是为了简化后面计算中的数学**（求导时平方的 2 与 $\frac{1}{2}$ 正好抵消）。取平方则是为了去掉根号，让目标函数变成光滑的二次函数（凸二次规划）。这两处改动都不改变最优解的位置。

4.4 第三步：拉格朗日乘子法

这是一个**带约束的优化问题**，用**拉格朗日乘子（Lagrange multipliers）**把约束吸收进目标函数。拉格朗日函数为：

$$\mathscr{L}(w, b, \alpha) = \tfrac{1}{2}\|w\|^2 - \sum_{i=1}^{N} \alpha_i [\, y_i(w \cdot x_i + b) - 1 \,]$$

其中 $\alpha_i \geqslant 0$ 是拉格朗日乘子。

> 课程网页备注：拉格朗日乘子是一个便利的数学工具，但**超出本课程范围**；感兴趣可参阅 Bishop 附录 E。考试不会要求你从零推导拉格朗日理论，但**要求你能看懂并复述这条推导链**。

4.5 第四步：求鞍点（对 w 和 b 求偏导并令其为零）

对 w 求偏导：

$$\partial \mathscr{L} / \partial w = w - \sum_{i=1}^{N} \alpha_i y_i x_i = 0 \quad \Rightarrow \quad \mathbf{w = \sum_{i=1}^{N} \alpha_i y_i x_i}$$

对 b 求偏导：

$$\partial \mathscr{L} / \partial b = -\Sigma_{i=1}^{N} \alpha_i y_i = 0 \quad \Rightarrow \quad \boldsymbol{\Sigma_i \alpha_i y_i = 0}$$

★ **考点** 第一个式子极其重要：**w 是训练样本的线性组合**，系数是 $\alpha_i y_i$。$\alpha_i = 0$ 的样本对 w 完全没有贡献——这就是「支持向量决定解」的数学来源。

4.6 第五步：代回得到对偶形式

把 $w = \Sigma \alpha_i y_i x_i$ 代回拉格朗日函数，得到问题的**对偶形式（dual form）**：

$$\max_{\alpha} \left[\Sigma_{i=1}^{N} \alpha_i - \tfrac{1}{2} \Sigma_{i=1}^{N} \Sigma_{j=1}^{N} \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) \right]$$

约束条件：

$$\Sigma_{i=1}^{N} \alpha_i y_i = 0 \quad \text{且} \quad \alpha_i \geqslant 0$$

★ **考点** **为什么要用对偶形式？** 因为在对偶形式中，数据**只以点积 $(x_i \cdot x_j)$ 的形式出现**。这正是核函数可以插入的位置——把点积替换成 $K(x_i, x_j)$ 就得到了核 SVM。原始形式里 w 是一个整体，无法直接替换。**对偶形式是核技巧的前提。**

4.7 求解与支持向量（选读）

求对偶问题的最优 α 通常使用**二次规划（quadratic programming）**技术，即优化带线性约束的二次目标。细节超出本课程范围。

解出 α 后：

- **大多数 α_i 都会是零**；对应**非零 α_i 的点就是支持向量**。这些点离决策边界最近，定义了最优超平面。
- 权重向量由 $w = \Sigma_{i=1}^{N} \alpha_i y_i x_i$ 计算得到。
- **只有支持向量（$\alpha_i > 0$）对 w 有贡献**，这意味着**决策边界完全由这些关键点决定**。

4.8 推导链一览（背这一条就够）

```
最大化最小带标签几何间隔
   arg max (1/‖w‖) · min y(w·x+b)
        ↓  利用缩放自由度，令支持向量的函数间隔 = 1
规范形式：y(w·x+b) ≥ 1，几何间隔 = 1/‖w‖
        ↓  间隔与 ‖w‖ 成反比
min ½‖w‖²   s.t.  y(w·x+b) ≥ 1
        ↓  拉格朗日乘子吸收约束
ℒ = ½‖w‖² − Σ α[y(w·x+b) − 1]
        ↓  ∂ℒ/∂w = 0 → w = Σ αyx ；∂ℒ/∂b = 0 → Σ αy = 0
        ↓  代回
对偶：max Σα − ½ΣΣ αα yy (x·x)   s.t. Σαy = 0, α ≥ 0
        ↓  数据只以点积出现
核技巧：(x_i·x_j) → K(x_i, x_j)
```

4.9 软间隔在对偶中的变化

引入松弛变量后，对偶问题的形式几乎不变，**唯一的差别是 α 多了一个上界**：

$$\text{硬间隔：} \alpha_i \geqslant 0 \quad \rightarrow \quad \text{软间隔：} 0 \leqslant \alpha_i \leqslant C$$

这就是 C 在对偶视角下的含义：**单个样本影响力的天花板**。详见第二部分 Tutorial 扩展题。

5. 可靠的 SVM 工作流程（第 6 页）

副标题：**缩放和验证，跟分类器本身一样重要。**

顺序为什么重要：每一步产生的信息会被下一步用到；顺序错了会导致数据泄漏（leakage）、无效比较（invalid comparisons）或难以排查的错误。

步骤 1 • Scale features（缩放特征）

Fit scaling on training data inside a pipeline

- **为何 SVM 特别需要缩放：**SVM 从头到尾都在算距离。间隔是 $1/\|w\|$，RBF 核算的是 $\exp(-\gamma\|x-z\|^2)$。尺度大的特征会主导距离计算。
- **fit on training data：**缩放器的均值与标准差只能从训练集算，否则测试集信息会通过统计量渗入训练过程（数据泄漏）。
- **inside a pipeline：**交叉验证时每一折会自动只用该折训练部分 fit 缩放器，强制正确。

步骤 2 • Choose baseline（选择基线）

Start with a linear kernel and a reproducible split

线性核最简单、最快、参数最少（只有 C）；若线性核已够好就没有理由上更复杂的核。固定随机种子（random_state）保证划分可复现，模型之间的比较才有意义。

步骤 3 • Inspect support（检查支持情况）

Check margin violations and support-vector count

支持向量占比过高通常意味着模型在硬啃噪声、边界过于复杂（往往是 C 或 gamma 太大）；占比很低说明数据分得较干净。这一步是在读懂模型，不是只看准确率。

步骤 4 • Tune carefully（谨慎调参）

Cross-validate C, kernel and gamma; test once

用交叉验证在训练数据上挑参数；**测试集只用一次**。反复用测试集比较模型会使其变成另一个验证集，报告分数系统性偏高。

6. 软间隔与超参数（第 7 页）

6.1 松弛变量 ξ

约束放松为 $y_i f(x_i) \geqslant 1-\xi_i$，$\xi_i \geqslant 0$。课件限定：在最优解处取**所需的最小松弛**，等价于

$$\xi_i = \max(0, 1 - y_i f(x_i))$$

（即 hinge loss 的形式。）由此可直接推出四档取值：

ξ 的值	对应 y f(x)	点的位置
$\xi = 0$	$y f(x) \geqslant 1$	在支持超平面上或之外（on / outside）
$0 < \xi < 1$	$0 < y f(x) < 1$	分类正确，但在间隔内部（inside）
$\xi = 1$	$y f(x) = 0$	**正好在决策边界上（boundary）**
$\xi > 1$	$y f(x) < 0$	被误分类（misclassified）

> ★ **考点 ξ = 1 是高频考点。**此时 $y f(x) = 0$，点恰好落在决策边界上。不能说它「在正确的一侧」——因为它**不在任何一侧**，$f(x) = 0$ 意味着没有严格的类别归属。

6.2 目标函数与 C

$$\min \ \tfrac{1}{2}\|w\|^2 + C\sum_i \xi_i$$

两项在拔河：$\frac{1}{2}\|w\|^2$ 想让 $\|w\|$ 小（间隔 $2/\|w\|$ 大）；$C\Sigma\xi$ 想让违规少。

- **C 大** → 重罚违规 → 宁可缩窄间隔也要迁就每个点 → 边界复杂、可能过拟合
- **C 小** → 容忍违规 → 更宽、正则化更强的间隔 → 可能欠拟合

> ★ **考点** 记忆口诀：**C 大 = 严格 = 窄间隔；C 小 = 宽松 = 宽间隔。**

6.3 核函数与 gamma

- **Kernel**：定义相似度，也就决定了边界可能的形状。

- **Gamma**：控制 RBF 影响**有多局部**。γ 大 → 影响范围窄 → 边界更弯曲；γ 小 → 影响范围广 → 边界更平滑。

C 与 gamma 都能提升复杂度但机制不同：C 管「容不容忍违规」，gamma 管「单个点的影响范围」。二者必须一起用交叉验证搜索。

6.4 SVC 的输出不是概率

> **SVC scores are not probabilities unless calibrated.**
> `decision_function` 给出的是 f(x)，一个带符号的实数，可以任意大，不在 [0,1] 内。要得到概率必须做**校准（calibration）**，如 Platt scaling（sklearn 中 `probability=True`，代价是额外的内部交叉验证）。

7. 核函数的选择（第 8 页）

选核原则：**先做线性基线，只有当验证结果支持时才增加灵活度。**

7.1 核函数的定义

$$K(x, z) = \phi(x)^T \phi(z)$$

核函数直接给出两点在特征空间中的点积，而不必真的算出 ϕ(x)。本质是**在特征空间中度量相似度**。

7.2 高维的威力：为什么需要核

设想数据排列成一个圆环：圆内的点属于一类，圆外的点属于另一类。**在二维中，没有任何直线能清楚地分开这些点。**解决办法是把数据**映射（map）**到更高维的空间，在那里它可能变得线性可分——例如把 2D 数据映射到 3D，在高维中一个线性边界就能干净地分开数据。

但**显式地把每个数据点映射到高维可能计算代价极高**。如果数据本身维度已经很多，映射到更高维就变得不切实际。

> ★ **考点** 核技巧让我们能**在一个隐式的特征空间中工作，而不必显式计算每一个变换后的坐标**。核直接计算变换后观测之间的**内积**，它起到**相似度度量**的作用。**核一般不是一个距离。**

7.3 核技巧手算验证（degree-2 多项式核）

二次多项式核的一般形式（含常数 c）：

$$K(x_i, x_j) = (x_i \cdot x_j + c)^2$$

c = 0 时对应的映射把 2D 点 $[a_1, a_2]$ 变成 3D 向量：

$$\phi([a_1, a_2]) = [a_1^2, \sqrt{2}\, a_1 a_2, a_2^2]$$

取 $x_i = [1, 2]$，$x_j = [3, 4]$。

Step 1 · 用核技巧直接算

$$K(x_i, x_j) = (1 \cdot 3 + 2 \cdot 4)^2 = (3 + 8)^2 = 11^2 = \mathbf{121}$$

Step 2 · 显式映射后再算点积

- $\phi(x_i) = [1^2, \sqrt{2} \cdot 1 \cdot 2, 2^2] = \mathbf{[1, 2\sqrt{2}, 4]}$
- $\phi(x_j) = [3^2, \sqrt{2} \cdot 3 \cdot 4, 4^2] = \mathbf{[9, 12\sqrt{2}, 16]}$

$$\phi(x_i) \cdot \phi(x_j) = (1 \cdot 9) + (2\sqrt{2} \cdot 12\sqrt{2}) + (4 \cdot 16) = 9 + (2 \cdot 12 \cdot 2) + 64 = 9 + 48 + 64 = \mathbf{121}$$

> **结论：**映射后的点积与核计算**都得到 121**。核计算在**没有显式构造变换向量**的情况下得到了这个值。这能省下构造特征的工作，但**核 SVM 在大数据集上训练仍可能昂贵，因为它可能需要大量的成对核求值。**

> ★ **考点** 注意 $2\sqrt{2} \times 12\sqrt{2} = 2 \cdot 12 \cdot (\sqrt{2} \cdot \sqrt{2}) = 2 \cdot 12 \cdot 2 = \mathbf{48}$，这一步是手算最容易出错的地方。

7.4 三个核的对比

核	公式	边界形状	需调参数
Linear 线性	$K = x^T z$	直的（straight）	C
Polynomial 多项式	$K = (\gamma x^T z + r)^d$	**全局曲线**（global curve）	C, d, γ, r
RBF 径向基	$K = \exp(-\gamma \| x-z \|^2)$	**局部曲线**（local curve）	C, γ

- **线性核**保留原始特征表示，边界是超平面，适合大致线性可分的情形。
- **多项式核**表达次数不超过 d 的特征交互，产生弯曲但相对全局的边界——改变数据一角，整条曲线都受影响。
- **RBF 核**通过距离衡量相似度：近点相似度高，远点趋近 0，所以每个样本影响是局部的。

7.5 RBF 的 gamma

- **γ 小 = 更平滑（smoother）** —— 每个观测影响范围广
- **γ 大 = 更复杂（more complex）** —— 影响范围窄，边界可能很不规则

若 RBF 边界太不规则，应**调查把 γ 调小**，并用验证证据支持这个改动。

7.6 两条操作纪律

1. **比较多项式核或 RBF 之前必须先缩放特征**，因为二者直接依赖 $x^T z$ 或 $\| x-z \|$。
2. 在训练数据上交叉验证 C、gamma 与其他核参数，预处理与 SVM 放在同一 pipeline；测试集只在最终模型选定后用一次。

> **奥卡姆剃刀：**更复杂的核不是更好的核，只有验证证据能证明它更好时才值得用。

8. 概念检查点与第一部分总结（第 9–10 页）

8.1 五个核心量

概念	要点
Score 分数	$f(x) = w^T x + b$，一切的起点
Distance 距离	有符号 $f(x)/\| w \|$；无符号 $\|f(x)\|/\| w \|$。**间隔条件用 y f(x)，不是单靠距离**
Support vector	非零对偶权重；定义边界的点（违规的点也算）
Soft margin	松弛允许违规，代价由 C 控制
Kernel	把 $x_i^T x_j$ 替换为 $K(x_i, x_j)$

> ★ **考点** 核技巧必须在**对偶形式**里做：只有对偶形式中数据仅以点积出现，才有替换的余地。原始形式里 w 是整体，无法直接换核。

8.2 五条关键要点（第 10 页左栏）

1. SVM 仍然从数值特征开始（不能直接吃类别变量或文本）
2. 间隔要求仔细区分分数与距离
3. 决定边界的是支持向量，不是每一个点
4. C 控制拟合与间隔的权衡
5. 核函数使非线性决策边界成为可能

8.3 进 Tutorial 前的五条注意事项（评分点）

1. **保持 ±1 标签约定的一致性**（不是 {0,1}，否则 $y \cdot f(x)$ 的判断规则全部失效）
2. **计算距离时把范数写出来**
3. **说明是一侧间隔还是完整宽度**
4. **仔细展开多项式特征的点积**
5. **解释结果，而不只给最终数字**

9. 其他经典机器学习算法

本课程深入学习的算法只占机器学习版图的一部分。这一章介绍另外三种广泛使用的**监督学习**算法：**朴素贝叶斯（Naive Bayes）**、**K 近邻（KNN）**和**决策树（Decision Trees）**。

> **考试范围提示：**这是**概念性概述**。课程**不要求你实现或推导**这些算法，但你应当**认出它们的主要思想，并理解它们何时有用**。
>
> 学习目标：① 描述三者的核心思想；② 指出每个算法用什么信息做预测；③ 比较它们的主要优势、局限与适用场景。

9.1 朴素贝叶斯（Naive Bayes）

一种基于贝叶斯定理的**概率分类算法**。它估计一个观测属于每个类别的概率，并预测**后验概率最大**的类别。

$$P(C_k \mid x) \propto P(C_k) \prod_{j=1}^{d} P(x_j \mid C_k)$$

「**朴素（naive）**」指的是它的简化假设：**在已知类别的条件下，各特征是条件独立的**。这个假设常常不现实，但模型仍然可能表现得出奇地好。

预测流程

1. 估计每个类别的先验概率 $P(C_k)$
2. 估计每个特征值在各类别内的可能性 $P(x_j \mid C_k)$
3. 对新观测把这些量组合起来
4. 预测后验概率最大的类别

手算例题：两个词指示器的垃圾邮件判定

设教学数据给出 P(spam) = 0.4，P(not spam) = 0.6。对一封同时包含 "offer" 和 "win" 的邮件，设：

- P(offer | spam) = 0.8，P(win | spam) = 0.5
- P(offer | not spam) = 0.1，P(win | not spam) = 0.05

在条件独立假设下，**未归一化分数**为：

$$\text{spam：} 0.4 \times 0.8 \times 0.5 = \mathbf{0.16}$$
$$\text{not spam：} 0.6 \times 0.1 \times 0.05 = \mathbf{0.003}$$

跨类别归一化：

$$P(\text{spam} \mid \text{offer, win}) = 0.16 / (0.16 + 0.003) \approx \mathbf{0.9816}$$

后验较大者获胜，因此**预测为 spam**。

> ★ **考点 Check your understanding：**分母包含**两个类别的分数**（不要只除以自己那一项）。结果依赖于所估计的概率和条件独立假设；**相关的词会让置信度产生误导**。这些是示意性概率，不是来自真实邮件数据集的测量值。

优势

- 训练和预测都快
- 在文本等高维数据上表现良好
- 提供类别概率估计
- 即使训练集相对较小也可能表现不错

局限

- 条件独立假设常常不现实
- 概率估计可能校准不佳
- 某个特征值在某类别中从未出现会产生零概率，除非使用平滑（smoothing）

9.2 K 近邻（K-nearest neighbours, KNN）

KNN 从训练数据中**最相似的已标注样本**做预测。它**不在训练时学习显式的方程或决策树**；它只是存储训练样本，把大部分工作放到**请求预测时**才做。

分类时，KNN 找出 K 个最近的训练样本并采用**多数投票**；回归时通常**对它们的目标值取平均**。

预测流程

1. 选择邻居数 K
2. 计算新观测到各训练观测的距离
3. 选出 K 个最近的观测
4. 用它们的标签投票（分类），或用它们的目标值形成预测（回归）

例：要给一朵新的花分类，KNN 可能找出测量值最相似的五朵训练花；若其中四朵属于同一物种，KNN 就预测该物种。

优势

- 简单直观
- 对决策边界的形状几乎不做假设
- 能表示复杂的非线性边界
- 分类与回归都能用

局限

- 训练集大时**预测很慢**
- 结果强烈依赖 K 与距离度量的选择
- **特征通常必须缩放**，否则数值大的特征会主导距离
- 高维空间中性能会退化，因为距离变得不具信息量——**维度灾难（curse of dimensionality）**

9.3 决策树（Decision Trees）

决策树通过一连串关于输入特征的 **if–then 问题**做预测。每个**内部节点**包含一个问题，每条**分支**代表一个答案，每个**叶节点**给出一个预测。

训练时，算法搜索能使所得分组**更同质（homogeneous）**的分裂。分类树常用 **Gini 不纯度**或**熵（entropy）**衡量；回归树使用平方误差之类的度量。

预测流程

1. 从树根开始
2. 评估当前节点的特征条件
3. 沿对应分支走下去
4. 持续直到到达一个叶节点
5. 用该叶节点存储的类别或数值作为预测

例：一棵简单的贷款审批树可能先问申请人收入是否超过某阈值，再问负债水平或还款历史；走过这些问题的路径产生最终预测。

优势

- 树较小时易于可视化与解释
- 能处理非线性关系与特征交互
- **几乎不需要特征缩放**
- 分类与回归都能用

局限

- 深树容易过拟合训练数据
- 数据的微小变化可能产生完全不同的树（不稳定）
- 贪心分裂不保证全局最优树
- 单棵树往往不如随机森林、梯度提升树等集成方法准确

9.4 一览对比表

算法	主要预测思想	缩放重要吗	主要优势	主要局限
朴素贝叶斯	组合类别先验与特征似然	**有时**	对文本与高维数据快速有效	强独立性假设
KNN	在邻近的已标注样本中投票或取平均	**需要**	简单灵活	预测慢、对距离敏感
决策树	沿学到的 if–then 特征分裂走下去	**通常不需要**	可解释、能处理非线性交互	容易过拟合且不稳定

> ★ **考点** 这张表是**最可能出选择题**的地方，尤其是「缩放重要吗」这一列：**KNN 需要（依赖距离），决策树通常不需要（只比较阈值），朴素贝叶斯有时需要**。对照 SVM——SVM 也依赖距离，所以**必须**缩放。

9.5 如何在它们之间选择

不存在普遍最优的算法。合理的选择取决于数据、预测任务以及不同错误的代价：

- 需要**快速的概率基线**，尤其是文本或计数数据 → 考虑**朴素贝叶斯**
- 观测之间的**相似性有意义**，且数据集不太大也不太高维 → 考虑 **KNN**
- **可解释性与特征交互**重要 → 考虑**决策树**

在真实项目中，候选模型应当使用**适当的验证流程与评估指标**来比较。**模型选择应基于来自未见数据的证据，而不只是训练表现。**

> **Key takeaway：**朴素贝叶斯用**概率**推理，KNN 从**邻近样本**推理，决策树通过**学到的 if–then 规则**推理。它们提供了处理监督学习的三种不同思路，也说明了为什么**模型选择应当反映数据的结构与应用的需求**。

第二部分 · Tutorial 完整解答

Question 1：分数、距离与间隔条件

题目： $f(x) = x_1 + x_2 - 3$，$w = (1, 1)$，四个带标签的点：(1,2,+1)、(2,3,+1)、(3,3,−1)、(1,1,−1)。用 $y\,f(x) \geqslant 1$ 作为规范间隔条件。

第一步永远是算范数： $\|w\| = \sqrt{(1^2 + 1^2)} = \sqrt{2} \approx 1.414$

x	y	f(x)	有符号距离	y f(x)	判定	ξ
(1,2)	+1	0	0	0	边界上；违规	1
(2,3)	+1	2	$\sqrt{2}$	2	正确；间隔外	0
(3,3)	−1	3	$3/\sqrt{2}$	−3	误分类；违规	4
(1,1)	−1	−1	$-1/\sqrt{2}$	1	正确；在支持超平面上	0

逐点说明

- **(1,2)：** $f = 1+2-3 = 0$，点在边界上，没有严格类别归属，说「分类正确」不准确。
- **(2,3)：** $f = 2$，有符号距离 $2/\sqrt{2} = \sqrt{2}$（注意化简到最简形式），$y\,f = 2 \geqslant 1$ 完全安全。
- **(3,3)：** $f = 3 > 0$ 判为正类，但真实标签为 −1，故误分类。有符号距离 $+3/\sqrt{2}$ 与带标签间隔 -3 符号相反，正是误分类的标志。
- **(1,1)：** $f = -1$，$y\,f = 1$ 恰好卡在边界值，$\xi = 0$，不算违规。

> **答案纸的两句总结（必背）：** f(x) 的符号给出点在哪一侧；f(x) = 0 是边界点，没有严格的类别归属。规范间隔条件用的是 y f(x)。

Question 2：间隔宽度与 C 的作用

第 1 小问：w = (3, 4)

$\|w\| = \sqrt{(9 + 16)} = 5$

$$1/\|w\| = \mathbf{0.2} \quad \text{（一侧）} \qquad 2/\|w\| = \mathbf{0.4} \quad \text{（完整宽度）}$$

第 2 小问：模型 A vs 模型 B

模型 A：间隔更宽，但有三个训练违规。模型 B：间隔更窄，无训练违规。

> **答案纸原文逻辑：** 较小的 C 对违规的惩罚较轻，因此**倾向于偏好模型 A** 的更宽间隔。较大的 C 对违规惩罚更重，因此**在能够避免训练违规的情况下倾向于偏好模型 B**。

	模型 A	模型 B
$\frac{1}{2}\|w\|^2$ 项	小（间隔宽 → $\|w\|$ 小）	大
$C\Sigma\xi$ 项	大（3 个违规）	0（无违规）
C 小时总代价	更低✓	较高
C 大时总代价	较高	更低✓

> ★ **考点** 注意答案纸措辞谨慎：「tends to favour（倾向于偏好）」，并附条件「if avoiding training violations is possible」。答题时也应使用这种带条件的表述。

第 3 小问：为什么更宽的间隔不保证更好的测试表现

> **答案纸原文：** 更宽的间隔可以降低对小扰动的敏感性，但泛化能力还取决于**数据、缩放、核函数、噪声以及所选的超参数**。

- **数据：** 训练集是否代表真实分布
- **缩放：** 不同缩放方式改变 $\|w\|$ 的含义，间隔宽度不可跨缩放比较

- **核函数：**核选错了，再宽的间隔也是在错误的空间里量的
- **噪声：**违规点若是噪声，容忍反而更好；若是真实模式，强行容忍就是欠拟合
- **超参数：**C、gamma 直接决定模型复杂度

Question 3：验证核恒等式

特征映射：$\phi(x_1, x_2) = (x_1^2, \sqrt{2}\,x_1x_2, x_2^2)$

第 1 小问：具体计算

- $\phi(1, 1) = (1^2, \sqrt{2}\cdot 1\cdot 1, 1^2) = \mathbf{(1, \sqrt{2}, 1)}$
- $\phi(2, 2) = (2^2, \sqrt{2}\cdot 2\cdot 2, 2^2) = \mathbf{(4, 4\sqrt{2}, 4)}$

验证：$\phi(1,1)\cdot\phi(2,2) = 1\cdot 4 + \sqrt{2}\cdot 4\sqrt{2} + 1\cdot 4 = 4 + 8 + 4 = \mathbf{16}$；而 $x^Ty = 1\cdot 2 + 1\cdot 2 = 4$，$(x^Ty)^2 = \mathbf{16}$ ✓

（注意 $\sqrt{2}\times 4\sqrt{2} = 4\times 2 = 8$，这一步最易算错。）

第 2 小问：一般证明

$$\phi(x)\cdot\phi(y) = x_1^2y_1^2 + 2x_1x_2y_1y_2 + x_2^2y_2^2 = (x_1y_1 + x_2y_2)^2 = (x^Ty)^2$$

中间项来历：$(\sqrt{2}\,x_1x_2)(\sqrt{2}\,y_1y_2) = \mathbf{2}\,x_1x_2y_1y_2$。然后逆用完全平方公式 $a^2 + 2ab + b^2 = (a+b)^2$，其中 $a = x_1y_1$，$b = x_2y_2$。

> ★ 考点 **$\sqrt{2}$ 存在的唯一目的**就是让交叉项系数变成 2。若写成 (x_1^2, x_1x_2, x_2^2)，交叉项系数只有 1，恒等式**不成立**。这是本题最经典的失分点。

第 3 小问：核技巧避免了什么

> **答案纸原文：**核技巧避免了**显式构造变换后的特征**；它直接计算这些特征的点积。这可以让非常高维甚至**无限维**的特征映射变得可行。
>
> **但不保证训练便宜**，因为随着数据集增长，核方法仍可能需要大量的**成对核求值（pairwise kernel evaluations）**。

	便宜	昂贵
特征维度 d	✓ 与 d 几乎无关	
样本数量 n		✗ n×n 核矩阵，$O(n^2)$ 次成对求值，训练常在 $O(n^2)$~$O(n^3)$

> ★ 考点 **一句话总结：核技巧在特征维度上省钱，在样本数量上花钱。**预测也不便宜——$f(x) = \Sigma\,\alpha_iy_iK(x_i, x) + b$，支持向量越多预测越慢。

Extension：对偶系数 α 与上界 C

题目：在软间隔对偶中 $0 \leqslant \alpha_i \leqslant C$。解释为什么 $\alpha_i > 0$ 的样本是有影响力的支持向量，以及上界 C 如何把对偶问题与间隔违反联系起来。

为什么 α > 0 的样本决定边界

$$f(x) = \Sigma_i\,\alpha_i\,y_i\,K(x_i, x) + b$$

只有 α_i 非零的样本对最终决策函数有贡献，所以它们决定了边界。$\alpha_i = 0$ 的项在求和里直接变成 0，删掉该样本解也不变。这正是「nonzero dual influence」的含义。

上界 C 的作用

界 $\alpha_i \leqslant C$ 限制了单个样本能对解产生多强的影响。相当于给每个样本的「话语权」设了天花板：无论一个点多难分，它最多只能施加 C 那么大的影响。这是 SVM 对离群点相对稳健的机制来源。

	C 增大	C 减小
α 的上限	提高	降低
单个样本最大影响力	更大	更小
对违规的惩罚	更重	更轻
训练违规数量	更少	更多
间隔宽度	更窄	更宽
正则化强度	更弱	更强

C 在原始问题与对偶问题中是同一个参数、扮演一致的角色：原始问题里是违规的惩罚系数，对偶问题里是 α 的上界。

补充：α 的三种取值对应的位置（KKT 结论，高频考点）

α_i	点的位置	ξ_i	名称
$\alpha = 0$	在间隔之外，正确分类（$y f > 1$）	0	非支持向量
$0 < \alpha < C$	恰好在支持超平面上（$y f = 1$）	0	free / margin SV
$\alpha = C$	间隔违反（$y f < 1$），间隔内或被误分类	> 0	bounded SV

> ★ **考点** 回到 Tutorial Q1 的四个点：(1,1) 的 $y f = 1$，属于第二类；(1,2) 与 (3,3) 违规，属于第三类（α 顶到 C）；(2,3) 的 $y f = 2$，$\alpha = 0$，不是支持向量。**这张表把对偶变量 α、松弛变量 ξ 与几何位置三者完全对应起来。**

第三部分 · Practical 完整解答

学习目标与好习惯

学习目标：比较线性与非线性核；解释为什么缩放必须放在 pipeline 内部；把 C 与间隔违反、支持向量数量联系起来；用验证证据选设置、把测试数据留给最终评估；实现双参数验证搜索并重新拟合选定模型。

四条好习惯：① 预处理放在 pipeline 内部防止泄漏；② **一次只改一个建模选择**；③ 用验证数据选设置；④ 除准确率外还要检查边界形状与支持向量。

Setup：数据与三重划分

```
RANDOM_STATE = 42
X, y = make_moons(n_samples=360, noise=0.24, random_state=7)

X_trainval, X_test, y_trainval, y_test = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=RANDOM_STATE)

X_train, X_val, y_train, y_val = train_test_split(
    X_trainval, y_trainval, test_size=0.25, stratify=y_trainval,
    random_state=RANDOM_STATE)
```

数据	数量	用途	使用次数
train	216	拟合模型参数（w、b、α）	每个候选模型各一次
val	72	选择超参数（核、C、gamma）	反复使用
test	72	最终无偏评估	**只用一次**

- **360 → 288 / 72**，再 **288 → 216 / 72**。第二次的 `test_size=0.25` 是相对 288 而非 360。
- **noise=0.24：**制造类别重叠。若无噪声两月牙完全分开，任何非线性模型都能 100%，就看不出 C 与 gamma 的差别。
- **stratify=y：**分层抽样，保证每份数据两类比例与原始一致。
- **random_state=42：**可复现划分，所有模型用同一划分比较才公平。

> ★ **考点** 验证集有 72 个点，所以所有验证准确率都是 x/72：0.833333 = 60/72，0.916667 = 66/72，0.958333 = 69/72。

Q1：公平比较三种核

```
kernel_settings = {
    "linear":     {"kernel": "linear"},
    "polynomial": {"kernel": "poly", "degree": 3},
    "RBF":        {"kernel": "rbf", "gamma": "scale"},
}
for name, settings in kernel_settings.items():
    model = Pipeline([("scale", StandardScaler()),
                      ("svc", SVC(C=1, **settings))])
    model.fit(X_train, y_train)
```

- `gamma="scale"` 是 sklearn 默认值，γ = 1 / (n_features × X.var())。
- **C=1 写死在循环外**——一次只改一个建模选择，唯一变化的是核，这才叫公平比较。
- `named_steps["svc"].n_support_` 给出**每个类别**的支持向量数，`.sum()` 才是总数。

kernel	validation_accuracy	support_vectors
linear	0.8333（60/72）	73
polynomial	0.8056（58/72）	73
RBF	**0.9167（66/72）**	**71**

参考答案的推理链

1. **线性核明显欠拟合**——月牙是弯的，直线追不上，属模型容量不足。
2. **三次多项式在这个划分上也不见得更好**——它产生全局曲线，与月牙的局部弯曲结构不匹配。**更复杂 ≠ 更好。**
3. **RBF 是最强的候选**，因为验证准确率最高，**而且并不需要比其他核更多的支持向量**（71 < 73）——说明它不是靠记住更多点取胜。
4. 结论基于单一划分，**重复交叉验证会提供更有力的证据**。

Q2：检查决策边界

```
xx, yy = np.meshgrid(
    np.linspace(X[:,0].min()-0.5, X[:,0].max()+0.5, 240),
    np.linspace(X[:,1].min()-0.5, X[:,1].max()+0.5, 240))
labels = model.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
plt.contourf(xx, yy, labels, alpha=0.25, cmap="coolwarm")

support_indices = model.named_steps["svc"].support_
plt.scatter(X_train[support_indices,0], X_train[support_indices,1],
            facecolors="none", edgecolors="#F2C14E", linewidths=1.8, s=105)
```

决策边界没有解析式，所以铺一张 240×240 的网格，把每个格点交给模型预测再填色，交界处就是边界。

属性	返回什么
`n_support_`	每类的支持向量**个数**，如 [36, 35]
`support_`	支持向量在训练集中的**索引**
`support_vectors_`	支持向量的**坐标（缩放后）**

> ★ **考点** 代码用 `support_` 索引回 `X_train` 而不是直接用 `support_vectors_`，因为后者是**缩放后**的值，画在原始坐标系上会错位。同理 `model.predict` 调用的是整个 pipeline，网格点也会先经过 StandardScaler。

参考答案的四个要点

1. **欠拟合：**线性边界无法跟随两个弯曲的类别。
2. **不必要的复杂度：**非线性核能表示月牙几何，但**远离观测数据处的弯折不应被过度解读；单凭视觉上的复杂并不是更好泛化的证据**（那些区域模型只是在外推）。
3. **支持向量的作用：**金色圈标出的训练观测**落在、位于内部、或以其他方式帮助定义**已拟合的间隔和边界；非支持向量不直接决定最终解。
4. **不同的核识别出不同的支持向量集合**，因为它们以不同方式表示相似度和几何。支持向量不是数据的固有属性。

Q3：用验证数据调 C

固定 RBF 核（Q1 的结论），只变 C，候选 [0.1, 1, 100]（**对数尺度**，因为 C 的效果是乘性的）。

C	validation_accuracy	support_vectors
0.1	0.861（62/72）	**120**
1	**0.917（66/72）**	71
100	0.903（65/72）	**36**

两个方向不同的模式

- **支持向量数：单调下降 120 → 71 → 36。**C 小 → 惩罚轻 → 允许大量违规 → 违规点全是支持向量（α = C）。C=0.1 时 120/216 = 56% 的训练点是支持向量，说明间隔非常宽。
- **验证准确率：倒 U 形 0.861 → 0.917 → 0.903。**两端都不好，中间最好。

C	状态	症状
0.1	正则化过强 / 欠拟合	间隔太宽，模型太钝
1	平衡点	验证准确率最高
100	正则化过弱 / 偏过拟合	边界紧贴训练数据，准确率回落

> **参考答案：**更大的 C 更严厉地惩罚违反，因此可能由更少的观测决定一个更紧的拟合，**但最大的值并没有给出最好的验证结果**。这里选择 **C = 1**。
>
> 测试集不用于这个选择，因为**反复使用它会把评估信息泄漏到模型选择中**；它被保留用于一次最终的无偏检查。

Q4：同时搜索 C 和 gamma（编程挑战）

```
c_grid     = [0.1, 1, 10, 100]
gamma_grid = [0.1, 1, 10]
for C in c_grid:
    for gamma in gamma_grid:
        candidate = Pipeline([("scale", StandardScaler()),
                              ("svc", SVC(kernel="rbf", C=C, gamma=gamma))])
        candidate.fit(X_train, y_train)
        ...
search_results = search_results.sort_values(
    ["validation_accuracy", "support_vectors"],
    ascending=[False, True]).reset_index(drop=True)
best_c     = float(search_results.loc[0, "C"])
best_gamma = float(search_results.loc[0, "gamma"])
```

> ★ 考点 **两级排序：**主键验证准确率**降序**，次键支持向量数**升序**——即选最高准确率，**只在平局时才用「支持向量更少」作为决胜依据**。`reset_index(drop=True)` 是必须的，否则 `.loc[0]` 取到的不是第一行。

排名	C	gamma	验证准确率	支持向量
0	**1**	**1**	**0.958**	64
1	10	1	0.931	45
2	1	10	0.931	123
3	0.1	10	0.931	**216**
4	100	1	0.917	31
5	100	0.1	0.917	58
6	100	10	0.917	99
7	10	10	0.903	109
8	0.1	1	0.903	129
9	10	0.1	0.861	73
10	1	0.1	0.833	83
11	0.1	0.1	0.819	133

选出：C = 1, gamma = 1，验证准确率 0.958（69/72），64 个支持向量。

- 第 1–3 名都是 0.931，但支持向量分别是 45、123、216——**决胜规则真的起了作用**。
- 第 3 名 C=0.1, γ=10 的 **216 个支持向量 = 全部训练点**：C 极小加 γ 极大，模型极其臃肿，预测要算 216 次核。

C 与 gamma 的交互（本题核心洞察）

C \ gamma	0.1	1	10
0.1	0.819	0.903	0.931
1	0.833	**0.958**	0.931
10	0.861	0.931	0.903
100	0.917	0.917	0.917

> ★ 考点 三列走向完全不同：**γ = 0.1 时 C 越大越好**（边界太平滑，需靠大 C 硬拉）；**γ = 1 时 C = 1 最好**；**γ = 10 时反而小 C 更好**（边界已够弯，大 C 只会加剧过拟合）。这就是交互——若先固定 γ=0.1 单独调 C，会得出「C=100 最好」，之后再调 γ 就永远碰不到 (1, 1) 这个真正的最优点。

> **参考答案：**C 和 gamma 会相互作用；C 控制违规的惩罚，gamma 控制 RBF 影响的局部程度。**非常大的值会造成不必要的局部边界；非常小的值会过于平滑。**
>
> **两个局限：**① 同一批验证观测被反复比较（交叉验证会给出更稳定的估计）；② **任何有限的网格都不保证是普遍最优设置。**

Q5：最终无偏评估

```
final_model = Pipeline([("scale", StandardScaler()),
                        ("svc", SVC(kernel="rbf", C=best_c, gamma=best_gamma))])
final_model.fit(X_trainval, y_trainval)
final_test_accuracy = accuracy_score(y_test, final_model.predict(X_test))
# Final test accuracy: 0.958
```

> ★ **考点** **最容易写错的一行：** `fit(X_trainval, y_trainval)` 而不是 `fit(X_train, y_train)`。参数一旦选定，验证集就完成使命，可以合并回来，用全部 **288 个非测试点**重新训练。这**不构成泄漏**。此外用变量 `best_c` / `best_gamma` 而非硬编码数字，保证上下游一致。

> **参考答案：**在固定 C = 1 和 gamma = 1 之后，模型在所有非测试数据上重新拟合，达到约 **0.958** 的测试准确率。**测试结果只在建模选择被固定之后才使用。反复从测试表现中挑选设置，会把测试集变成验证数据，并使报告的估计变得乐观。**

如何看待两个 0.958：验证准确率 0.958 是 12 次比较中的**最大值**，天然乐观；测试准确率 0.958 是**一次评估**的结果，才是可信的泛化估计。两者相等属巧合（均为 69/72，样本都只有 72 个）。

Practical 总结与数据流

结论	对应的题
核函数控制决策边界的形状	Q1、Q2
C 控制间隔—违规的权衡	Q3
pipeline 防止预处理泄漏	贯穿全程
验证集选设置，测试集估计最终泛化能力	Q3、Q5
结构化的验证搜索能暴露超参数之间的交互	Q4

```
360 点 → 划分 216/72/72
  ↓
Q1  固定 C=1, 变核      → 选 RBF    (val 0.917)
  ↓
Q2  画边界 + 支持向量   → 理解 RBF 为什么赢
  ↓
Q3  固定 RBF, 变 C      → 选 C=1    (val 0.917)
  ↓
Q4  同时搜 C 和 gamma   → 选 (1, 1) (val 0.958)
  ↓
Q5  refit 到 288 点     → 测试 0.958  ← 只用这一次
```

> ★ **考点** 每一步**只改一个东西**，且每步结论成为下一步的固定前提。论述题问「你会怎么做模型选择」，照这个流程答即可。

附录 A · 公式速查表

名称	公式	备注
打分函数	$f(x) = w^{T}x + b$	感知机与 SVM 共用
范数	$\|w\| = \sqrt{(\Sigma w_j^2)}$	手算必须写出这一步
有符号距离	$f(x)/\|w\|$	保留在哪一侧
点到边界距离	$\|f(x)\|/\|w\|$	非负
带标签几何间隔	$\gamma_i = y_i f(x_i)/\|w\|$	分对为正
规范间隔条件	$y_i f(x_i) \geqslant 1$	用 y f(x)，不是距离
一侧间隔	$1/\|w\|$	相差一倍，务必标注
完整间隔宽度	$2/\|w\|$	
松弛变量	$\xi_i = \max(0, 1 - y_i f(x_i))$	hinge loss 形式
软间隔目标	$\min \frac{1}{2}\|w\|^2 + C\Sigma\xi_i$	两项拔河
原始优化问题	$\min \frac{1}{2}\|w\|^2$ s.t. $y_i(w \cdot x_i+b) \geqslant 1$	½ 是为简化求导
拉格朗日函数	$\mathscr{L} = \frac{1}{2}\|w\|^2 - \Sigma\alpha_i[y_i(w \cdot x_i+b) - 1]$	$\alpha_i \geqslant 0$
$\partial\mathscr{L}/\partial w = 0$	$w = \Sigma\alpha_i y_i x_i$	w 是样本的线性组合
$\partial\mathscr{L}/\partial b = 0$	$\Sigma\alpha_i y_i = 0$	对偶的等式约束
对偶目标	$\max \Sigma\alpha_i - \frac{1}{2}\Sigma\Sigma\alpha_i\alpha_j y_i y_j (x_i \cdot x_j)$	数据只以点积出现
对偶约束	硬间隔 $\alpha_i \geqslant 0$；软间隔 $0 \leqslant \alpha_i \leqslant C$	C 是影响力天花板
对偶决策函数	$f(x) = \Sigma\alpha_i y_i K(x_i, x) + b$	核技巧的落点
核定义	$K(x, z) = \phi(x)^{T}\phi(z)$	变换空间中的点积
线性核	$K = x^{T}z$	直边界
多项式核	$K = (\gamma x^{T}z + r)^{d}$	全局曲线
RBF 核	$K = \exp(-\gamma\|x-z\|^2)$	局部曲线
二次核恒等式	$\phi(x) \cdot \phi(y) = (x^{T}y)^2$，$\phi = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$	$\sqrt{2}$ 让交叉项系数为 2

y f(x) / ξ / α 三合一对照表

y f(x)	ξ	α	位置与判定
> 1	0	0	间隔之外，正确；**非**支持向量
= 1	0	0 < α < C	支持超平面上；free / margin SV
0 < y f < 1	0 < ξ < 1	= C	正确但在间隔内；违规；bounded SV
= 0	1	= C	**边界上，无类别归属**；违规
< 0	> 1	= C	误分类；违规

附录 B · 高频易错点清单

1. **分数 ≠ 距离 ≠ 间隔条件。**f(x) 是分数；f(x)/‖w‖ 才是距离；y f(x) 才是间隔条件。三个量三个用途，答题时必须分别写出并标名。
2. **一侧 1/‖w‖ vs 完整 2/‖w‖。**差一倍。题目问哪个答哪个，并明确标注你算的是哪个。
3. **ξ = 1 时点在决策边界上**，y f(x) = 0，**没有类别归属**，不能说「在正确的一侧」。
4. **y f(x) > 0 只代表分对，不代表没违规。**要 ⩾ 1 才安全。
5. **C 的方向：**C 大 = 严惩违规 = 窄间隔 = 弱正则；C 小 = 容忍违规 = 宽间隔 = 强正则。
6. **C 与准确率的关系不是单调的**（实测倒 U 形），但 **C 与支持向量数量是单调的**（C 增大，支持向量减少）。
7. **SVM 使用 ±1 标签，不是 {0,1}。**逻辑回归才是 {0,1}。用错标签体系，y f(x) 的全部判断规则失效。
8. **SVC 的 decision_function 不是概率**，除非做过校准。
9. **核技巧的代价：**省的是特征维度，费的是样本数量（n×n 核矩阵，成对求值）。**不保证大数据集上训练便宜。**
10. **特征映射里的 √2 不能漏**，否则交叉项系数变成 1，恒等式不成立。
11. **缩放必须在划分之后、在 pipeline 内部。**先对全数据缩放再划分 = 数据泄漏。
12. **测试集只用一次。**一旦用于选择，它就变成了验证集，报告的估计系统性偏高。
13. **最终 refit 要用 train + val（全部非测试数据）**，不是只用 train。这不构成泄漏。
14. **更复杂的核不一定更好**（本练习中多项式核 0.806 < 线性核 0.833）。
15. **支持向量多不代表模型好**，往往相反——可能在硬啃噪声。
16. **C 与 gamma 存在交互**，必须联合网格搜索，不能分别单独调。
17. **最大间隔不保证泛化。**泛化还取决于数据、缩放、核、噪声与超参数，最终要靠验证证据。
18. **边界图上远离数据处的弯折是外推**，不应被过度解读；视觉复杂不是泛化更好的证据。
19. **support_ 是索引，support_vectors_ 是缩放后的坐标**，画图时用索引回原始数据取坐标。
20. **硬间隔要求数据完美可分**，否则无解，必须用软间隔。
21. **函数间隔 ≠ 几何间隔。**规范形式下支持向量的函数间隔 y(w · x+b) = 1，但几何距离是 **1/‖w‖，不是 1。**
22. **几个样本点中的最小距离不自动等于 SVM 间隔**，除非这些点确实是已拟合 SVM 的支持向量。
23. **½‖w‖² 中的 ½ 与平方都只是为了数学方便**（求导抵消、去根号变凸二次），不改变最优解。
24. **核不是距离**，它是内积、是相似度度量。相似度高 ≠ 距离大。
25. **对偶形式是核技巧的前提**：只有对偶里数据以点积出现才能替换成 K。
26. **KNN 必须缩放，决策树通常不必，朴素贝叶斯有时需要。**三者容易混。
27. **朴素贝叶斯归一化时分母要包含全部类别的分数**，不能只除自己那一项。
28. **KNN 训练几乎不花时间，代价全在预测**；这与大多数模型相反。

附录 C · 术语中英对照

英文	中文	英文	中文
hyperplane	超平面	slack variable	松弛变量
margin	间隔	hinge loss	合页损失
maximum-margin separator	最大间隔分隔面	dual / primal	对偶 / 原始
signed distance	有符号距离	dual coefficient	对偶系数
canonical scaling	规范化缩放	kernel trick	核技巧
support hyperplane	支持超平面	feature map	特征映射
support vector	支持向量	Gram / kernel matrix	核矩阵
hard / soft margin	硬 / 软间隔	calibration	（概率）校准
margin violation	间隔违反	data leakage	数据泄漏
regularisation	正则化	pipeline	流水线
underfitting / overfitting	欠拟合 / 过拟合	stratified split	分层划分
generalisation	泛化	grid search	网格搜索
cross-validation	交叉验证	reproducible split	可复现划分
validation evidence	验证证据	extrapolation	外推
Lagrange multiplier	拉格朗日乘子	saddle point	鞍点
functional margin	函数间隔	geometric margin	几何间隔
quadratic programming	二次规划	inner product	内积
Naive Bayes	朴素贝叶斯	conditional independence	条件独立
posterior / prior	后验 / 先验	likelihood	似然
K-nearest neighbours	K 近邻	curse of dimensionality	维度灾难
decision tree	决策树	Gini impurity / entropy	基尼不纯度 / 熵
ensemble methods	集成方法	smoothing	平滑